# gaussian-issue

April 28, 2026

[ ]:
```
import os
import sys

CWD = os.getcwd()
PARENT_DIR = os.path.dirname(CWD)
sys.path.append(PARENT_DIR)
```

[ ]:
```
from pathlib import Path

import numpy as np
import pandas as pd

from src import _to_host, xp
from src.adaptive_clustering_utils import SurrogateMode
from src.problems.anisotropic_gaussian import aniso_gaussian
from src.surrogate import AFINNISurrogate


def generate_diagnostic_report(
    model_path: str, dim: int, domain: list, n_test: int = 50000
):
    print(f"\n{'='*60}")
    print(f"REPORT: {Path(model_path).name}")
    print(f"{'='*60}")

    # 1. Load Model
    surrogate = AFINNISurrogate.load_model(model_path)
    Nc = surrogate.X_src.shape[0]

    print(f"Model loaded: {Nc:,} centroids, mode={surrogate.mode}")

    # 2. Generate Test Data
    xp.random.seed(0)
    X_test = xp.random.uniform(domain[0], domain[1], size=(n_test, dim))
    f_true = aniso_gaussian(X_test)

    # 3. Extract Internal Prediction Data (k=1)
```

```
    # We use return_cluster_assignments=True to see exactly which model each point got
    preds, ung_mask, nn_idx = surrogate._predict_local(
        X_test,
        k_nn=1,
        coarse_factor=100,
        strict_trust_region=False,
        return_cluster_assignments=True,
    )

    # 4. Manual Calculation of Mahalanobis Distances (d_Q)
    centroids = surrogate.X_src[nn_idx[:, 0]]
    Qs = surrogate.Qs_src[nn_idx[:, 0]]
    f_mus = surrogate.f_src[nn_idx[:, 0]]
    dx = X_test - centroids
    # dist_sq = dx^T * Q * dx
    dist_sq = xp.einsum("ni,nij,nj->n", dx, Qs, dx)
    dist_q = xp.sqrt(dist_sq)

    # 5. Extract Gradient Norms (||A||)
    if surrogate.A_fits_src is not None:
        A_models = surrogate.A_fits_src[nn_idx[:, 0]]  # (N, fd, d)
        a_norms = xp.linalg.norm(A_models, axis=(1, 2))
    else:
        a_norms = xp.zeros(n_test)

    # 6. Separate populations
    in_mask = ~ung_mask
    out_mask = ung_mask

    # 7. Compile Statistics
    stats = {
        "Metric": [
            "Population (%)",
            "Avg d_Q (Trust Dist)",
            "Max d_Q (Trust Dist)",
            "Avg ||A|| (Gradient)",
            "Avg True F (Value)",
            "Avg f(mu) (Centroid Value)",
        ],
        "IN-Sample": [
            f"{xp.sum(in_mask)/n_test*100:.2f}%",
            f"{xp.mean(dist_q[in_mask]):.4f}",
            f"{xp.max(dist_q[in_mask]):.4f}",
            f"{xp.mean(a_norms[in_mask]):.2e}",
            f"{xp.mean(f_true[in_mask]):.4f}",
            f"{xp.mean(f_mus[in_mask]):.4f}",
```

```
        ],
        "OUT-Sample": [
            f"{xp.sum(out_mask)/n_test*100:.2f}%",
            f"{xp.mean(dist_q[out_mask]):.4f}" if xp.any(out_mask) else "N/A",
            f"{xp.max(dist_q[out_mask]):.4f}" if xp.any(out_mask) else "N/A",
            f"{xp.mean(a_norms[out_mask]):.2e}" if xp.any(out_mask) else "N/A",
            f"{xp.mean(f_true[out_mask]):.4f}" if xp.any(out_mask) else "N/A",
            f"{xp.mean(f_mus[out_mask]):.4f}" if xp.any(out_mask) else "N/A",
        ],
    }

    print(pd.DataFrame(stats).to_string(index=False))


if __name__ == "__main__":
    # Point the below list to your saved surrogate models.
    models = [
        "./results/sc26/anisotropic_gaussian/mpi_sz14_dim5_tol0.
↪1_ntrain185646_affine_cov_hess_L2_kmeans_ninit92_minpts371_stealTrue_v2True_surrogate_model
↪npz",
        "./results/sc26/anisotropic_gaussian/mpi_sz14_dim5_tol0.
↪005_ntrain185646_affine_cov_hess_L2_kmeans_ninit92_minpts371_stealTrue_v2True_surrogate_mod
↪npz",
    ]
    models = [
        "../results/sc26/anisotropic_gaussian/mpi_sz14_dim5_tol0.
↪1_ntrain185646_affine_cov_hess_Linf-local_kmeans_ninit92_minpts371_stealTrue_v2True_surroga
↪npz",
        "../results/sc26/anisotropic_gaussian/mpi_sz14_dim5_tol0.
↪005_ntrain185646_affine_cov_hess_Linf-local_kmeans_ninit92_minpts371_stealTrue_v2True_surro
↪npz",
    ]

    for m in models:
        try:
            generate_diagnostic_report(m, dim=5, domain=[-3.0, 3.0])
        except Exception as e:
            print(f"Could not process {m}: {e}")
```

```
/home/prajwal/github-repos/hatann/src/__init__.py:60: UserWarning: CuPy not
available; GPU acceleration will be disabled.
  warnings.warn(msg)


============================================================
REPORT: mpi_sz14_dim5_tol0.1_ntrain185646_affine_cov_hess_Linf-
local_kmeans_ninit92_minpts371_stealTrue_v2True_surrogate_model.npz
============================================================
```

```
Model loaded: 156 centroids, mode=SurrogateMode.AFFINE_COV_HESS
                     Metric IN-Sample OUT-Sample
            Population (%)    99.99%      0.01%
      Avg d_Q (Trust Dist)    0.5877     1.0000
      Max d_Q (Trust Dist)    1.0000     1.0000
      Avg ||A|| (Gradient)  1.45e-04   1.78e-07
        Avg True F (Value)    1.0000     1.0000
Avg f(mu) (Centroid Value)    1.0000     1.0000

============================================================
REPORT: mpi_sz14_dim5_tol0.005_ntrain185646_affine_cov_hess_Linf-
local_kmeans_ninit92_minpts371_stealTrue_v2True_surrogate_model.npz
============================================================
Model loaded: 18,694 centroids, mode=SurrogateMode.AFFINE_COV_HESS
                     Metric IN-Sample OUT-Sample
            Population (%)    98.35%      1.65%
      Avg d_Q (Trust Dist)    0.5864     1.8177
      Max d_Q (Trust Dist)    1.0000    14.0841
      Avg ||A|| (Gradient)  3.02e-04   1.28e-02
        Avg True F (Value)    1.0000     1.0000
Avg f(mu) (Centroid Value)    1.0000     1.0007
```

# 1 Analysis: Large errors for Anisotropic Gaussian

- Anisotropic Gaussian

$$f(x) = \beta + \exp\left(-\frac{1}{2}x^T\Sigma^{-1}x\right), \quad \beta = 1$$

- Surrogate Model

$$\hat{f}(x) = f(\mu) + \mathbf{A}(x-\mu)$$

- For points in the tail, the error is dominated by the linear term:

$$\begin{aligned} e(x) &= |\,[f(\mu) + \mathbf{A}(x-\mu)] - f(x)| \\ &\approx |\,[f(\mu) + \mathbf{A}(x-\mu)] - \beta| \qquad \text{(since } f(x) \approx \beta \text{ in the tail)} \\ &\approx |\mathbf{A}(x-\mu)| \qquad \text{(since } f(\mu) \approx \beta) \\ &\leq \|\mathbf{A}\| \cdot \|x-\mu\| \qquad \text{(Cauchy-Schwarz)} \end{aligned}$$

## 1.1 Table - $5D$ Anisotropic Gaussian

Metrics - Avg $d_{\|\cdot\|_T}$ : Avg distance of query point to the closest cluster center in the trust region norm. - Max $d_{\|\cdot\|_T}$ : Max distance of query point to the closest cluster center in the trust region norm. - Avg $\|A\|$ : Avg norm of the linear term of the cluster's surrogate model. - Avg True $f$ (Value) : The average true function value at the query points. - Avg $f(\mu)$ (Centroid Value) : The average function value at the cluster centroids.

**Large Tolerance ($r_{\text{tol}} = 0.1$; `Linf-local`; $N_c = 156$; $e = 5.22E{-}3$)**

| Metric | IN | OUT |
|---|---|---|
| % Population | 99.99% | 0.01% |
| Rel $L_2$ Error | 6.89E-04 | 6.17E-02 |
| Avg $d_{\|\cdot\|_T}$ | 0.5877 | 1.0000 |
| Max $d_{\|\cdot\|_T}$ | 1.0000 | 1.0000 |
| Avg $\|A\|$ | 1.45e-04 | 1.78e-07 |
| Avg True $f$ | 1.0000 | 1.0000 |
| Avg $f(\mu)$ | 1.0000 | 1.0000 |

**Small Tolerance ($r_{\text{tol}} = 0.005$; `Linf-local`; $N_c = 18,694; e = 1.11E{-}2$)**

| Metric | IN | OUT |
|---|---|---|
| % Population | 98.35% | 1.65% |
| Rel $L_2$ Error | 3.89E-05 | 6.14E-02 |
| Avg $d_{\|\cdot\|_T}$ | 0.5864 | 1.8177 |
| Max $d_{\|\cdot\|_T}$ | 1.0000 | 14.0841 |
| Avg $\|A\|$ | 3.02e-04 | 1.28e-02 |
| Avg True $f$ | 1.0000 | 1.0000 |
| Avg $f(\mu)$ | 1.0000 | 1.0007 |

- Large Tolerance $r_{\text{tol}} = 0.1$
  - $N_c = 156$, clusters are large. In the tails, the fitted gradient $\mathbf{A}$ is effectively zero ($1.78 \times 10^{-7}$), reflecting the flat geometry.
  - Since 99.99% of the population is **IN**, the global error $e = 5.22E{-}3$ is dominated by the **IN** error, which is small ($6.89E{-}4$).
- Small Tolerance $r_{\text{tol}} = 0.005$
  - $N_c = 18,694$, clusters are tiny and slender (look at representative example from 2D case).
  - In the tails, an **OUT** point is often assigned to a nearest neighbor that is still on the Gaussian's slope, with a non-trivial gradient ($\|\mathbf{A}\| \approx 1.28 \times 10^{-2}$).
  - Additionally, the extrapolation distance $\|x - \mu\|$ is now massive (avg $d_{\|\cdot\|_T} = 1.82$), with the maximum being 14.08, which multiplies the "overfitted" gradient $\|\mathbf{A}\|$, resulting in large errors ($6.14E{-}2$) for the **OUT** points.
  - Finally, since a larger portion of the population is **OUT** (1.65%), the global error $e = 1.11E{-}2$ is now dominated by the **OUT** error, which is much worse than the large tolerance case.